



THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE
LINGUAGEM PARA CORREÇÃO E FEEDBACK DE
EXERCÍCIOS DE POO**

LAVRAS – MG

2026

THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE LINGUAGEM PARA
CORREÇÃO E FEEDBACK DE EXERCÍCIOS DE POO**

Trabalho de Conclusão de Curso apresentado
à Universidade Federal de Lavras, como
parte das exigências do Curso de Ciência da
Computação, para a obtenção do título de
Bacharel

Prof. Dr. Julio César Alves
Orientador

**LAVRAS – MG
2026**

**Ficha catalográfica elaborada pela Coordenadoria de Processos Técnicos da Biblioteca
Universitária da UFLA, com dados informados pelo(a) próprio(a) autor(a).**

Martins, Thallys Henrique

Aplicação e Avaliação de Grandes Modelos de Linguagem
para Correção e Feedback de Exercícios de POO / Thallys Hen-
rique Martins. 6^a ed. rev., atual. e ampl. – Lavras : UFLA,
2026.

23p. : il.

Tese(doutorado)–Universidade Federal de Lavras, 2026.

Orientador: Prof. Dr. Julio César Alves .

Bibliografia.

1. Inteligência Artificial. 2. Educação. 3. POO. 4. Feedback
Automatizado. I. Universidade Federal de Lavras. II. Título.

CDD 808.066

THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE LINGUAGEM PARA
CORREÇÃO E FEEDBACK DE EXERCÍCIOS DE POO**

**APPLICATION AND EVALUATION OF LARGE LANGUAGE MODELS FOR
CORRECTION AND FEEDBACK OF OOP EXERCISES**

Trabalho de Conclusão de Curso apresentado
à Universidade Federal de Lavras, como
parte das exigências do Curso de Ciência da
Computação, para a obtenção do título de
Bacharel

APROVADA em Data de Defesa.

Prof. MSc. Antônio Banca Um	UFM
Prof. DSc. João Banca Dois	FCO
Profa. Esp. Eliza Banca Três	BELMIS
Prof. Esp. Zorbison Uberplax IV	UFO

Prof. Dr. Julio César Alves
Orientador

**LAVRAS – MG
2026**

Espaço reservado à epígrafe.

(Autor Desconhecido)

RESUMO

A avaliação de exercícios práticos na disciplina de Programação Orientada a Objetos (POO) é fundamental para o aprendizado, mas exige um alto esforço do docente, especialmente em relação à consistência e clareza do feedback. Ferramentas de correção automática (como o JavaCorrige) são valiosas, mas muitas vezes limitadas na qualidade do retorno fornecido ao estudante. Com o avanço das Large Language Models (LLMs), torna-se viável a criação de um sistema de correção que ofereça feedback personalizado e sofisticado, superando as limitações dos sistemas tradicionais. A abordagem adotada neste projeto será a utilização da biblioteca LangGraph para criar um sistema de correção desacoplado, garantindo a flexibilidade técnica para comparar e alternar o modelo de IA (LLM) utilizado (ex: Gemini, OpenAI), o que é vital para a longevidade e validade da pesquisa.

Palavras-chave: programação orientada a objetos; grandes modelos de linguagem; langgraph; feedback pedagógico; avaliação de código.

ABSTRACT

The evaluation of practical exercises in Object-Oriented Programming (OOP) is fundamental for learning but demands high effort from instructors, particularly concerning consistency and feedback clarity. Automated correction tools (such as JavaCorrige) are valuable yet often limited in the quality of the feedback provided to the student. With the advancement of Large Language Models (LLMs), it becomes feasible to create a correction system that offers personalized and sophisticated feedback, overcoming the limitations of traditional systems. The approach adopted in this project involves using the LangGraph library to create a decoupled correction system, ensuring technical flexibility to compare and switch the AI model (LLM) used (e.g., Gemini, OpenAI), which is vital for the longevity and validity of the research.

Keywords: object-oriented programming; large language models; langgraph; pedagogical feedback; code evaluation.

SUMÁRIO

1	INTRODUÇÃO	7
1.1	Objetivos	8
1.2	Estrutura do Texto	8
2	REFERENCIAL TEÓRICO	9
2.1	Ensino de Programação Orientada a Objetos (POO)	9
2.2	Modelos de Linguagem de Grande Escala (LLMs)	9
2.3	Correção Automática de Código	9
2.4	IA como Tutor e Orquestração Pedagógica	10
2.5	Eficiência Computacional e Escalabilidade	10
2.6	A Biblioteca Langgraph	11
3	METODOLOGIA	12
3.1	Contexto da Pesquisa	12
3.2	Planejamento do Desenvolvimento	12
3.3	Metodologia de Desenvolvimento	13
3.4	Arquitetura do Sistema e Fluxo de Orquestração	14
3.5	Interface e Integração de Dados	14
4	RESULTADOS	15
4.1	Análise dos Estudos de Caso: O Papel do Gabarito	15
4.2	Análise Qualitativa de Submissões Reais	15
4.2.1	Consistência Pedagógica e Clean Code	16
4.2.2	Limitações Técnicas e Sensibilidade Visual	16
4.3	Discussão das Convergências Teóricas	16
	APÊNDICES	17
A	APÊNDICE B - NOVA VERSÃO DO PROMPT	19

1 INTRODUÇÃO

A crescente demanda por profissionais qualificados na área de tecnologia tem impulsionado a busca por métodos de ensino de programação mais escaláveis e eficientes. No contexto do ensino superior, as disciplinas de Programação Orientada a Objetos (POO) representam um marco na formação do estudante, exigindo a transição do pensamento puramente algorítmico para uma visão baseada em design, abstração e arquitetura de sistemas. No entanto, a avaliação qualitativa de exercícios nesse paradigma apresenta desafios significativos.

O processo de fornecer feedback detalhado e personalizado é uma tarefa que consome um volume considerável de tempo do docente. De acordo com Pereira e Mello (2025), a falta de um retorno pedagógico imediato e consistente pode prejudicar o ciclo de aprendizagem do discente, especialmente quando os erros cometidos não são apenas sintáticos, mas conceituais, envolvendo princípios fundamentais do paradigma, como encapsulamento e coesão (Cipriano; Alves, 2023).

Ferramentas de correção automática tradicionais, embora úteis para validar se um código compila ou passa em testes unitários, frequentemente falham em oferecer uma análise qualitativa profunda. Estas ferramentas costumam atuar como validadores estruturais, focando no “que” o código faz e negligenciando o “como” ele foi projetado. Com a recente popularização dos Grandes Modelos de Linguagem (LLMs), abre-se uma nova fronteira para a automação pedagógica, permitindo a geração de feedbacks que se assemelham à análise realizada por um tutor humano.

Entretanto, a integração direta de LLMs em sistemas de ensino esbarra em obstáculos técnicos, como a tendência desses modelos a gerar informações imprecisas (alucinações) e a necessidade crítica de supervisão humana para garantir a sustentabilidade pedagógica (Pereira; Mello, 2025). Além disso, a rápida evolução tecnológica torna arriscado o desenvolvimento de sistemas dependentes de um único provedor de inteligência artificial.

Este trabalho propõe a aplicação e avaliação de um sistema de feedback automatizado construído sobre uma arquitetura desacoplada. Utilizando o framework LangGraph para orquestração de estados e o modelo Gemini 2.5 Flash via SDK oficial. A pesquisa foca na capacidade de alternar entre diferentes modelos generativos, garantindo a longevidade da ferramenta e avaliando como a presença de uma solução de referência (gabarito) altera a profundidade qualitativa do feedback fornecido ao estudante de POO.

1.1 Objetivos

O objetivo deste trabalho é desenvolver e avaliar um protótipo de sistema de correção de código para exercícios de POO utilizando a biblioteca LangGraph, com foco na garantia de flexibilidade técnica e na qualidade do feedback gerado por IA. Para isso, os seguintes objetivos específicos foram estabelecidos:

- Desenvolver o fluxo principal de correção e análise utilizando a biblioteca LangGraph, garantindo a flexibilidade técnica para alternar e comparar diferentes Modelos de Linguagem Grande (LLMs).
- Automatizar a geração de anotações estruturais e gabaritos de correção do JavaCorrige por meio da IA, reduzindo o esforço do docente.
- Implementar um módulo de IA para enriquecer o relatório de resultados, transformando notas quantitativas em feedback qualitativo e claro para os estudantes.
- Utilizar a IA para realizar a correção propriamente dita do código, e integrar seus resultados com a correção estrutural do JavaCorrige para compor uma avaliação final mais completa.
- Validar a eficácia, precisão e consistência das correções e dos feedbacks gerados pela IA em um conjunto de exercícios reais de Programação Orientada a Objetos.
- Analisar e documentar os custos, as limitações e as licenças das LLMs disponíveis para garantir a viabilidade econômica do sistema a longo prazo.

1.2 Estrutura do Texto

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta o referencial teórico que fundamenta a pesquisa realizada. Nele, são discutidos os principais conceitos, teorias e trabalhos relacionados ao tema abordado, proporcionando uma base sólida para a compreensão do problema de pesquisa e a justificativa da metodologia adotada. A revisão da literatura é organizada de forma a destacar as contribuições mais relevantes e identificar lacunas que a presente pesquisa busca preencher.

2.1 Ensino de Programação Orientada a Objetos (POO)

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2.2 Modelos de Linguagem de Grande Escala (LLMs)

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2.3 Correção Automática de Código

Historicamente, o feedback automatizado em ambientes educacionais tem se concentrado em aspectos quantitativos, como a correção de respostas objetivas ou a validação de código por meio de testes unitários. Embora essas abordagens garantam agilidade, elas limitam-se a informar se um resultado está correto ou incorreto, falhando em fornecer a orientação peda-

gógica necessária para que o discente compreenda a origem do seu erro conceitual. Com o advento das LLMs, torna-se possível realizar uma análise semântica e qualitativa que atinja um nível de feedback que se aproxima do oferecido por um tutor humano.

A viabilidade técnica dessa evolução é validada por estudos como o de Wang (2023), que demonstrou ser possível automatizar o feedback docente com alta fidelidade ao utilizar modelos especializados. A arquitetura proposta alcançou uma pontuação METEOR (*Metric for Evaluation of Translation with Explicit Ordering*) superior a 96%, o que indica que as respostas geradas são quase indistinguíveis do julgamento de um professor especializado. O uso da métrica METEOR é fundamental para essa conclusão, pois ela avalia a qualidade do texto considerando sinônimos e a coesão lógica, garantindo que o sistema ofereça um suporte pedagógico robusto e confiável para o aprendizado de programação.

2.4 IA como Tutor e Orquestração Pedagógica

De acordo com Pereira e Mello (2025), o uso de métodos manuais de avaliação em cursos de larga escala frequentemente resulta em atrasos na correção, o que impacta negativamente a motivação e a participação dos discentes. Os autores defendem que as LLMs oferecem uma solução para este problema ao possibilitarem respostas em tempo real, garantindo que o retorno pedagógico seja imediato e relevante para o processo de aprendizagem.

No entanto, um dos maiores desafios pedagógicos reside em evitar que a IA forneça a resposta pronta, comprometendo o esforço cognitivo do aluno. Jacobs e Jaschke (2023) demonstram que, ao utilizar modelos avançados como o GPT-4, é possível mitigar quase integralmente essa limitação, fornecendo sugestões e orientações sem expor a solução final. Os autores ressaltam que, para manter essa eficácia, é vital evitar o envio de informações contraditórias à IA (como conflitos entre o código do aluno e o gabarito), o que reforça a necessidade de fluxos de dados controlados por orquestradores para garantir a integridade do papel de tutor.

2.5 Eficiência Computacional e Escalabilidade

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam

tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2.6 A Biblioteca Langgraph

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

3 METODOLOGIA

Neste capítulo, apresentam-se os procedimentos, técnicas e materiais utilizados para o desenvolvimento e validação do sistema de correção automatizada. O objetivo é detalhar o percurso metodológico e as decisões técnicas que fundamentam a construção do protótipo, garantindo a transparência e a replicabilidade dos experimentos.

3.1 Contexto da Pesquisa

A pesquisa classifica-se como um estudo aplicado de natureza tecnológica. O contexto da investigação reside na disciplina de Programação Orientada a Objetos (POO) do curso de Ciência da Computação da Universidade Federal de Lavras (UFLA), utilizando submissões de códigos reais de discentes como base para os experimentos.

3.2 Planejamento do Desenvolvimento

O planejamento para a construção do sistema foi estruturado em marcos sequenciais, visando garantir que a lógica pedagógica fosse preservada em todas as camadas técnicas. As etapas principais que nortearam o desenvolvimento foram:

- **Levantamento de Requisitos:** A partir de discussões com o orientador e da identificação de dificuldades recorrentes no ensino de Programação Orientada a Objetos (POO), definiram-se os requisitos funcionais prioritários. Entre eles, destacam-se a automação do processo de análise e correção para reduzir o esforço docente na elaboração de feedbacks individuais e detalhados para cada estudante, além da implementação de um módulo de inteligência artificial via orquestração de estados. Este módulo é projetado para processar submissões e converter requisitos técnicos em um relatório qualitativo, estruturado em formato JSON, que fornece um retorno claro e instrutivo.
- **Definição da Arquitetura de Instrução:** Planejou-se uma arquitetura baseada em LLMs para realizar a análise lógica e pedagógica do código-fonte. Para viabilizar essa abordagem, adotou-se a fragmentação das instruções em arquivos Markdown independentes, o

que facilita a manutenção do sistema e permite que o docente ajuste critérios pedagógicos sem a necessidade de manipular o código-fonte da aplicação.

- **Prototipagem da Orquestração de Estados:** Utilizou-se a biblioteca LangGraph para desenvolver o fluxo principal de análise. Esta escolha visou garantir a flexibilidade técnica necessária para alternar e comparar diferentes modelos de linguagem, permitindo que o sistema não fique dependente de um único fornecedor e possa evoluir conforme novas tecnologias surjam.
- **Escolha de Tecnologias de Suporte:** Após a análise de viabilidade técnica e econômica, selecionaram-se ferramentas já estabelecidas na indústria para garantir o rigor dos experimentos, como o Gemini 3 Flash para inferência, o LangGraph para orquestração e o google-java-format para a padronização do código Java.

3.3 Metodologia de Desenvolvimento

O desenvolvimento do protótipo seguiu um ciclo iterativo e incremental, pautado pela experimentação contínua das capacidades de inferência do modelo de linguagem. Os procedimentos adotados para a construção e refinamento da ferramenta foram divididos nas seguintes etapas:

- **Ciclos de Refinamento de Prompt:** A construção das instruções contidas nos arquivos Markdown não foi estática. Realizaram-se sucessivas rodadas de teste utilizando submissões reais para calibrar a sensibilidade da IA. Ajustaram-se os arquivos `persona.md` e `system_prompt.md` para evitar que o modelo apresentasse comportamentos excessivamente permissivos ou rigorosos de forma injustificada. É importante ressaltar que o enunciado do exercício atua como um componente dinâmico do *prompt*. Como a IA utiliza as restrições descritas no problema para realizar sua análise, a clareza e a precisão terminológica do enunciado influenciam diretamente a acurácia do *feedback*.
- **Normalização de Dados de Entrada:** Durante o desenvolvimento, identificou-se que a presença de comentários extensos e espaços redundantes elevava o consumo de *tokens* sem agregar valor à análise. Assim, incorporou-se ao método uma etapa de limpeza via *Regex*, visando a redução do custo operacional e a otimização da janela de contexto.

Complementarmente, utilizou-se a formatação via *linter* (`google-java-format`), pois observou-se que variações na formatação do código Java (como espaçamentos inconsistentes) impactavam diretamente a previsibilidade do *feedback*. Esse procedimento garante que a entrada para a LLM seja padronizada, minimizando ruídos estruturais e aumentando a precisão da avaliação.

- **Avaliação de Consistência e Estabilidade:** Para validar o desenvolvimento, os mesmos exercícios foram submetidos repetidamente para verificar a variabilidade das respostas.

3.4 Arquitetura do Sistema e Fluxo de Orquestração

Como o sistema foi projetado, grafo de estados...

3.5 Interface e Integração de Dados

Interface, funcionamento do sistema, processamento, como o código é enviado, como o feedback é retornado...

4 RESULTADOS

Este capítulo apresenta a análise dos experimentos realizados com o protótipo, avaliando o impacto da solução de referência (gabarito) na eficácia do *feedback* gerado. A análise foi dividida em testes controlados (Estudos de Caso I e II) e uma análise qualitativa de submissões reais, buscando evidenciar as convergências entre a atuação da IA e as necessidades pedagógicas de POO.

4.1 Análise dos Estudos de Caso: O Papel do Gabarito

A experimentação inicial com os exercícios “Nave” e “Casa” permitiu observar como a ancoragem em um gabarito altera a natureza da correção. Notou-se que, na ausência de uma solução de referência, a IA tende a atuar como um analisador funcional autônomo. Nesse estado, o modelo foca primordialmente na existência técnica dos componentes solicitados no enunciado e em erros de compilação.

Em contrapartida, a introdução do gabarito elevou o nível da avaliação. No cenário do exercício “Nave”, a presença da solução de referência permitiu que a IA identificasse não apenas a funcionalidade, mas a **equivalência técnica**. Por exemplo, o sistema pôde inferir que um código era idêntico à versão inicial não modificada, algo que uma análise sem gabarito interpretou apenas como ausência de implementação.

No exercício “Casa”, observou-se uma sofisticação maior no controle de estado (uso do booleano `porDoSolRealizado`). Com o suporte do gabarito, a IA demonstrou flexibilidade pedagógica ao classificar divergências paramétricas triviais — como pequenas variações de coordenadas — como aceitáveis, mantendo o foco na lógica de orquestração de eventos. Como aponta o referencial teórico sobre o ensino de POO, o foco deve residir na arquitetura e não apenas na sintaxe, o que valida a abordagem híbrida proposta.

4.2 Análise Qualitativa de Submissões Reais

Após a validação controlada, o sistema foi submetido a códigos reais de discentes, revelando nuances críticas sobre o comportamento do modelo Gemini.

4.2.1 Consistência Pedagógica e Clean Code

Um dos resultados mais significativos foi a capacidade da IA de aplicar conceitos de *Clean Code* e coesão. Observou-se que o modelo identifica a “mistura de responsabilidades”, como o caso recorrente de alunos que implementam a movimentação de um objeto dentro de métodos de controle ambiental. Além disso, a IA demonstrou rigor sobre o encapsulamento, sugerindo a remoção de métodos *getters* desnecessários que não foram solicitados, reforçando o conceito de interface mínima em POO.

Todavia, emergiu o desafio da “lógica no construtor”. A IA identificou corretamente como má prática a chamada de métodos públicos dentro do construtor, reforçando que este deve ser restrito à inicialização de estado. Essa descoberta converge com as melhores práticas da indústria mencionadas na fundamentação deste trabalho.

4.2.2 Limitações Técnicas e Sensibilidade Visual

A análise qualitativa também revelou divergências indesejadas. Identificou-se uma **vulnerabilidade à indentação**, onde o modelo alucinou erros de sintaxe (como classes não fechadas) devido a chaves desalinhadas no código do aluno. Este dado é relevante para a discussão, pois sugere que o pré-processamento via *linter* (citado na Metodologia) é indispensável para garantir a previsibilidade do sistema.

Outro ponto de discussão é o “Rigor Geométrico”. A IA tendeu a penalizar divergências estéticas (cores e tamanhos) que não eram restritas pelo enunciado. Esse comportamento evidencia a necessidade de refinamento no *System Prompt*, ensinando o modelo a diferenciar erros de arquitetura graves de escolhas criativas irrelevantes, evitando um *feedback* desencorajador ao estudante.

4.3 Discussão das Convergências Teóricas

APÊNDICES

APÊNDICE A – PROMPT DE CORREÇÃO UTILIZADO

Abaixo apresenta-se a estrutura exata das System Instructions utilizadas para orquestrar o comportamento da IA durante o processo de correção:

Listing 1 – Instrução de Sistema para Correção Automatizada

```
Você é um Professor de Programação Orientada a Objetos (POO) da UFLA
. Avalie o código Java do aluno considerando o enunciado. Forneça
feedback construtivo e educativo, focado em princípios de POO (
Encapsulamento, Herança, Lógica).
Sua resposta DEVE ser um JSON válido, SEM TEXTO ANTES OU DEPOIS, e
seguir EXATAMENTE este schema:
{
  "avaliacao": "Certo" | "Errado" | "Parcialmente Certo",
  "justificativa": string,
  "dica_correcao": string,
  "sugestao_correcao": string
}
No campo 'sugestao_correcao', coloque o código Java corrigido e
inclua comentários explicativos sempre que possível para ajudar o
aluno a entender as correções e boas práticas. Não inclua mensagens
de parabéns ou explicações fora do código. Se a sugestão envolver
mais de um arquivo, separe cada arquivo usando exatamente o padrão:
// --- ARQUIVO INÍCIO: NomeDoArquivo.java ---
(código do arquivo, com comentários explicativos se necessário)
// --- ARQUIVO FIM: NomeDoArquivo.java ---
No campo 'dica_correcao', coloque as orientações e dicas para o
aluno corrigir o código, sem incluir o código em si. Use ```java
apenas no prompt, não na resposta JSON. Responda integralmente em
português. Não adicione explicações fora do JSON.
```

Quando uma solução de referência (gabarito) é fornecida ao sistema, a instrução abaixo é concatenada ao prompt para garantir a ancoragem técnica:

Listing 2 – Instrução Adicional para Ancoragem por Gabarito

Avalie o código do aluno COMPARANDO DIRETAMENTE com o gabarito acima . Considere o gabarito como a única solução correta. Aponte diferenças, similaridades e se o código do aluno está igual, melhor ou pior que o gabarito. Siga a estrutura rígida definida no System Instruction.

APÊNDICE B – NOVA VERSÃO DO PROMPT DE CORREÇÃO UTILIZADO

Abaixo apresenta-se a estrutura exata das System Instructions utilizadas para orquestrar o comportamento da IA durante o processo de correção (nova versão):

Listing 3 – Prompt do Persona para Avaliação Automatizada

```
Você é um instrutor sênior de engenharia de software com mais de 15
anos de experiência em Java e design orientado a objetos. Adote um
tom claro, construtivo e pedagógico.
```

Listing 4 – Prompt do Sistema para Avaliação Automatizada

```
Siga estas regras rígidas ao avaliar código de aluno.

## Regras Rígidas

### JSON obrigatório

Sua resposta deve ser estritamente um JSON válido que satisfaça o
schema requerido e nada mais - sem texto livre antes ou depois.

Exemplo de shape (use exatamente estas chaves):
{
  "avaliacao": "Certo" | "Errado" | "Parcialmente Certo",
  "justificativa": "",
  "dica_correcao": "",
  "sugestao_correcao": ""
}

### Campos (regras específicas)

- sugestao_correcao: deve conter APENAS código Java corrigido - sem
comentários explicativos ou texto adicional. Para múltiplos arquivos
, separe exatamente assim (apenas conteúdo):

// --- ARQUIVO INÍCIO: NomeDoArquivo.java ---
<conteúdo do arquivo>
```

```
// --- ARQUIVO FIM: NomeDoArquivo.java ---
```

- justificativa: descrição técnica clara do porquê da avaliação (vinculada ao enunciado). Seja conciso e foque nos pontos relevantes.
- dica_correcao: instruções pedagógicas para o aluno corrigir (SEM incluir código).

```
### Erros e dados sensíveis
```

- Se detectar dados sensíveis (chaves, senhas, PII) no código, NAO reproduza o conteúdo sensível. Em justificativa indique sucintamente a presença e o impacto, sem expor os dados.

```
### Regras de comportamento do modelo
```

- Assuma que o código recebido já foi pré-processado: comentários removidos e formatado com google-java-format quando possível.
- Observação: se o enunciado exigir que o aluno adicione comentários, NAO penalize a ausência desses comentários, pois o pré-processamento os remove. Em vez disso, avalie a presença e o comportamento dos métodos esperados com base na semântica e nos efeitos observáveis do código.
- NAO inclua qualquer texto fora do JSON. Se for impossível produzir o JSON solicitado, retorne:

```
{ "avaliacao": "Erro", "justificativa": "<motivo>", "dica_correcao": "", "sugestao_correcao": "" }
```

- Se for fornecida a lista {{CONCEITOS_A_AVALIAR}}, avalie somente esses conceitos e ignore comentários sobre outros.

```
## Regra especial sobre gabarito e saída textual
```

- O gabarito do professor é um EXEMPLO de solução, não um molde obrigatório de formatação. NÃO penalize diferenças de formato/estilo quando a solução estiver correta em comportamento/semântica.
- Compare explícita e estritamente a saída textual (ex.: `System.out.println`) somente quando o enunciado exigir saída textual exata. Caso contrário, priorize corretude, tratamento de casos e aderência ao enunciado.
- Na justificativa, avalie se a solução demonstra entendimento do enunciado (trata casos, respeita restrições, implementa a lógica pedida). NÃO avalie nem penalize decisões criativas de formatação, posicionamento visual, nomes de variáveis ou escolhas estéticas quando elas NÃO impactarem a correção, segurança ou os requisitos do enunciado. Comente sobre diferenças apenas quando elas violarem restrições explícitas do enunciado ou influenciarem comportamento/saída do programa.

Linguagem e estilo

- Responda em Português (pt-BR).
- Use termos objetivos e técnicos; evite digressões.

Prioridades de correção

1. Correção/semântica (o programa atende o enunciado?)
2. Segurança e robustez
3. Legibilidade e estilo

Exemplos e verificações rápidas

- NÃO inclua trechos de código com informações sensíveis; sinalize a presença sem expor o conteúdo.

Princípios de revisão

- Seja específico: indique arquivos exatos e forneça exemplos concretos.
- Forneça contexto: explique POR QUE algo é um problema e seu impacto.
- Sugira soluções: mostre código corrigido quando aplicável.
- Seja construtivo: foque em melhorar o código, não em criticar o autor.
- Reconheça boas práticas quando presentes.
- Seja pragmático: nem toda sugestão precisa ser implementada imediatamente.
- Agrupar comentários correlatos: Evite múltiplos comentários sobre o mesmo tópico ou erro repetitivo.

Incorpore esses princípios ao preencher justificativa e dica_correcao - a justificativa deve conter a referência precisa e o impacto; a dica_correcao deve resumir próximos passos pedagógicos.

Siga estritamente estas regras; qualquer desvio deve resultar em avaliacao: "Erro" com justificativa explicando o problema.

